

TimeTracker — Feature Implementation Plan

Step-wise specification for AI work-verification reporting, HR (Horilla-style) modules, multi-team tracking, HR-assigned tasks, and day-to-day screenshot review.

Prepared: 11 June 2026 | Stack: Rust / Axum / PostgreSQL / SQLx · Next.js / TypeScript / Tailwind / shadcn · Tauri 2 desktop · Cloudflare R2 / MinIO · Gemini 2.5 Flash

0. Context and conventions

This plan extends the existing platform. Already implemented (STEP 0–10): role-based auth (employee / project_manager / hr), immutable UTC time intervals with a local-first SQLite sync queue, presence and idle detection with statuses **working / idle / break / meeting / not_working / not_logged_in**, screenshot capture with metadata-only storage and short-lived presigned URLs, the daily screenshot sampler (analysis_jobs / analysis_job_samples), and the Vision AI analyzer (analysis_results) comparing screenshots to assigned Linear tickets with Gemini 2.5 Flash.

Every feature below follows the project's standard workflow (CLAUDE.md): **1)** design schema → **2)** migration → **3)** repository → **4)** service → **5)** API route → **6)** frontend → **7)** tests → **8)** docs. Cross-cutting rules apply throughout: RBAC enforced server-side, all timestamps stored in UTC, SQLx compile-time-checked queries with migrations, audit logging of sensitive actions, presigned short-lived URLs (storage keys never exposed), and no secrets sent to clients.

Feature 1 — AI screenshot analysis report on the HR dashboard

Goal. Aggregate the per-screenshot AI verdicts (already produced in STEP 10) into a per-employee, per-day **report** that scores how well the day's activity matched the employee's assigned ticket(s)/task(s), and let HR review it on the dashboard.

Schema and migration

Table	Key columns	Notes
analysis_reports	id, user_id→users, day (DATE), job_id→analysis_jobs, total_analyzed, aligned_count, partially_count, not_aligned_count, inconclusive_count, alignment_score (0–100), summary_text, model, created_at	One row per (user, day). UNIQUE(user_id, day). Built from analysis_results.

Steps

- 1 Add migration 00xx_analysis_reports.sql with the table above (UNIQUE(user_id, day), FK cascade on user/job).
- 2 Repository db/analysis_reports.rs: upsert(report), get(user_id, day), list(scope, day_range).
- 3 Service report_service.rs: aggregate analysis_results for the day's job → counts + **alignment_score** = weighted(aligned=1, partial=0.5, not_aligned=0, inconclusive excluded) × 100; then call Gemini (text) to compose a short natural-language **summary_text** from the per-shot rationales.
- 4 Trigger: run report generation at the end of analyze_day, and via the scheduled daily job (Feature sequencing §7), so reports refresh idempotently.

- 5 API: GET /admin/users/:id/report?day= (HR / PM team-scoped), GET /admin/reports?day=&team= (roster of reports), GET /me/report?day= (employee's own).
- 6 Frontend (HR dashboard): a **Reports** view — per-employee report card showing the alignment score, a verdict-breakdown bar/donut, the day's sampled screenshots with their individual verdicts, and the AI summary. Drill-down from the team table.
- 7 Tests: aggregation math (counts & score), threshold/exclusion of inconclusive, RBAC gating; acceptance: given a day with analysed screenshots, an HR user sees a stored, scored report.

Feature 2 — Exclude meeting-mode screenshots from analysis

Goal. Screenshots captured while the employee is in **meeting** mode must be stored and viewable but never sampled or analysed by the AI.

Steps

- 1 Tag capture context: add a captured_status column to screenshots (working / meeting / break / ...) via migration; default 'working' for existing rows.
- 2 Desktop: include the current presence status/kind when posting screenshot metadata (screenshot .rs / upload notify).
- 3 Sampler (STEP 9): tighten eligibility so only captured_status = 'working' screenshots enter the bucket selection — meeting/break shots are skipped.
- 4 Analyzer (STEP 10): defensive guard — never send a non-working screenshot to Gemini even if passed in.
- 5 Tests: a day with mixed working/meeting shots samples only working ones; meeting shots produce no analysis_results row. Acceptance: meeting screenshots are excluded from all analysis while remaining visible in the gallery (Feature 3).

Feature 3 — Day-to-day screenshot & summary view with filtering

Goal. Present screenshots and their analysis summary in a per-day format; applying a filter (date / employee / team / verdict) shows the matching summary.

Steps

- 1 Backend: day-grouped listings — GET /admin/users/:id/screenshots?day= and GET /me/screenshots?day=, each returning presigned view URLs plus the per-shot verdict (joined from analysis_results) and a **meeting** flag.
- 2 Filtering params on the report/listing endpoints: day or from/to, team, verdict, user; the daily summary recomputes for the active filter.
- 3 Frontend: a day navigator (calendar/date-picker) on both the employee and HR dashboards. Selecting a day shows that day's screenshots (meeting ones badged “not analysed”) plus that day's summary/report. Filter controls refresh the summary in place.
- 4 Tests: listings are correctly bucketed by day in UTC→local boundary; filters return only matching rows; meeting shots shown but marked. Acceptance: choosing a date or filter yields the relevant screenshots and the matching summary.

Feature 4 — Multi-team tracking, team selection before timer, team summary

Goal. An employee may belong to multiple teams; before starting the timer they pick the team they are working for from a dropdown; HR/admin assign teams; the admin dashboard gains a **Team Summary** showing the whole team's activity and each member's individual summary.

Schema and migration

Table	Key columns	Notes
teams	id, name, description, created_at	Team catalogue.
user_teams	user_id→users, team_id→teams, PRIMARY KEY(user_id, team_id)	Many-to-many membership (one employee → many teams).
intervals (alter)	+ team_id→teams (nullable)	Tags each tracked interval with the team it was logged under.

Steps

- 1 Migration: create teams and user_teams; add nullable team_id to intervals. (Supersedes the single users.team_id for membership.)
- 2 Repositories: db/teams.rs (CRUD, list_members), membership add/remove; extend interval insert to carry team_id.
- 3 Admin/HR API + UI: create/edit teams, assign/unassign employees (an employee can be added to several teams). Extend the Manage Users page.
- 4 Employee dashboard: a **required team dropdown** (populated from the employee's memberships) shown before **Start**; the chosen team_id is passed to the start-tracking command and stamped on every interval of that session. If the employee has exactly one team, preselect it.
- 5 Admin **Team Summary** section: GET /admin/teams and GET /admin/teams/:id/summary returning team-level aggregates (total worked, status breakdown, who is active now) plus each member's individual totals; UI shows the team roll-up and expandable per-member summaries.
- 6 Tests: multi-team membership, interval team tagging, team aggregate math, PM scope. Acceptance: an employee in 2+ teams selects one before starting; the admin sees both the combined team summary and each individual.

Feature 5 — HR-assigned manual tasks analysed as tickets by the AI

Goal. HR can add a task directly to any employee's dashboard. It is not a real Linear ticket, but it is fed to the Vision AI as task context so screenshots are analysed against it just like a ticket.

Schema and migration

Table	Key columns	Notes
manual_tasks	id, user_id→users (assignee), created_by→users (HR), title, description, status (open/done), created_at, updated_at	HR-authored pseudo-tickets shown on the employee dashboard.

Steps

- 1 Migration: manual_tasks as above.
- 2 API: HR-only create/list/update/delete for a target employee (audit-logged); GET /me/tasks for the employee's own assigned tasks.

- 3 AI integration: the analyzer's context builder merges manual tasks with Linear tickets, mapping each task to the same context shape (id, title, state, labels=[], description_excerpt) with an id prefix such as task:<uuid>; matched_ticket may then reference a manual task.
- 4 Frontend: employee dashboard lists assigned manual tasks alongside the tickets panel; HR UI to add/close tasks per employee.
- 5 Tests: task appears in context, can be matched, RBAC (only HR creates). Acceptance: an HR-added task is analysed and can be the matched item in a screenshot verdict / report.

Feature 6 — Horilla-style HR portal: Onboarding, Leave, Attendance, Payroll

These mirror the corresponding modules of the open-source **Horilla HRMS** (github.com/horilla/horilla-hr). Horilla is a Django/MySQL application; we reimplement the same feature set and workflows natively in our Rust + Next.js stack (we do not import Django code). Each sub-module follows the standard 8-step workflow and lives in the HR portal with HR-only configuration and employee self-service where applicable.

6A. Onboarding

Goal. Recruit-to-hire pipeline: candidates move through onboarding stages with a task checklist and document collection, then convert into employee accounts.

Schema

- candidates (id, name, email, role/position, source, stage_id, status, hired_at)
- onboarding_stages (id, name, sequence) — e.g. Applied, Interview, Offer, Onboarding, Hired
- onboarding_tasks (id, stage_id, title) and candidate_tasks (candidate_id, task_id, done, done_at)
- candidate_documents (id, candidate_id, doc_type, storage_key)

Steps

- 1 Migrations for the tables above; seed default stages and a default task checklist.
- 2 Repositories + service: stage transitions, checklist completion, document upload (presigned, Rule 5).
- 3 API (HR): candidate CRUD, move stage, toggle task, upload docs; **convert candidate** → **user** (creates the employee account, optionally triggers a reset-password / invite).
- 4 Frontend: a Kanban board of stages with candidate cards and per-candidate checklist + documents.
- 5 Tests + acceptance: a candidate progresses through all stages and is converted into an employee.

6B. Leave management

Goal. Leave types with per-employee balances, leave requests with an approval workflow, and a company holiday calendar.

Schema

- leave_types (id, name, paid, default_days, carry_forward)
- leave_allocations (id, user_id, leave_type_id, year, allotted_days, used_days)
- leave_requests (id, user_id, leave_type_id, start_date, end_date, days, reason, status [pending/approved/rejected], approver_id, decided_at)
- holidays (id, date, name) and optional company-wide leave days

Steps

- 1 Migrations + seed common leave types (casual, sick, earned) and the year's holidays.
- 2 Service: balance computation (allotted – used), overlap/holiday-aware day counting, approval workflow with audit logging.
- 3 API: employee request/cancel leave + view balance; manager/HR approve/reject; HR configure types, allocations, holidays.

- 4 Frontend: employee leave form + balance widget; HR/manager approvals queue; holiday calendar.
- 5 Tests + acceptance: an employee submits a request, balance reserves the days, an approver decides, and the balance/attendance reflect the outcome.

6C. Attendance

Goal. Daily attendance derived primarily from the existing interval engine (check-in = first start, check-out = last stop), with present/absent/half-day/leave status and reports.

Schema

- `attendance_days` (id, user_id, day, first_in, last_out, worked_seconds, status [present/absent/half_day/leave/holiday]) — cached roll-up of intervals
- `optional_shifts` (id, name, start_time, end_time, grace_minutes) for late-in / early-out flags

Steps

- 1 Service: build `attendance_days` from intervals per day; mark days covered by approved leave (Feature 6B) or holidays; compute late/early against the shift if configured.
- 2 Trigger: nightly roll-up job + on-demand recompute for a date range.
- 3 API: employee own attendance; HR/manager team attendance report (range, export).
- 4 Frontend: attendance calendar/table per employee and a team report; integrates leave + holidays.
- 5 Tests + acceptance: a worked day yields a present record with correct in/out and worked hours; an approved-leave day shows as leave, not absent.

6D. Payroll

Goal. Per-employee salary structure, periodic payroll runs that generate payslips (incorporating attendance / loss-of-pay and leave), and employee payslip access.

Schema

- `salary_structures` (id, user_id, basic, allowances_json, deductions_json, effective_from)
- `payroll_runs` (id, period_start, period_end, status, run_at, run_by)
- `payslips` (id, payroll_run_id, user_id, gross, deductions, net, lop_days, breakdown_json, generated_at)

Steps

- 1 Migrations for salary structures, payroll runs, payslips.
- 2 Service: for a period, pull attendance + approved leave to compute loss-of-pay, apply the salary structure (basic + allowances – deductions – LOP) → gross / deductions / net.
- 3 API (HR): configure salary; run payroll for a period; (re)generate payslips; employee GET `/me/payslips`. Payslip PDF export reuses this PDF tooling.
- 4 Frontend: HR payroll run screen + payslip list; employee payslip download.
- 5 Tests + acceptance: a payroll run for a period produces a payslip per active employee with correct net pay reflecting attendance and leave.

7. Suggested build sequence and dependencies

Ordered to unblock dependencies and ship value incrementally:

#	Feature	Why this order / depends on
1	F2 — meeting-mode exclusion + screenshot status tagging	Small; makes all downstream analysis correct. No dependencies.
2	F1 + F3 — daily report aggregation, HR report view, day-to-day filtering	Builds on STEP 10 + F2. Delivers the core HR-visible reporting.
3	F4 — multi-team, team selection, team summary	Independent; enriches reports/attendance with team attribution.

4	F5 — HR manual tasks → AI	Plugs into the analyzer's context builder (post-F1).
5	F6 — Horilla modules: Onboarding → Leave → Attendance → Payroll	Largest. Attendance reuses intervals (+F4 teams); Payroll depends on Attendance + Leave.

8. Cross-cutting requirements (apply to every feature)

- **RBAC:** employee / project_manager / HR (admin) enforced server-side; employees see only their own data, managers their team, HR everything.
- **Data:** UTC storage (convert in UI only); SQLx compile-time-checked queries; a migration per schema change; repository pattern; no `unwrap()` in production code.
- **Security:** audit-log sensitive actions (create/update/delete, approvals, payroll, screenshot access); short-lived presigned URLs only; secrets server-side and git-ignored.
- **Testing:** unit + integration tests per feature; >80% coverage on critical modules; live end-to-end verification before sign-off.
- **Deliverables per feature** (CLAUDE.md): file-tree changes, migration files, tests, and a short architecture note.

End of plan. This document is a specification of intended work; implementation of each feature will proceed step-by-step with verification, in the sequence of §7.